

西南石油大学

《金融经济数据挖掘》实验报告

金融新闻情绪量化、羊群效应构建与非线性预测分析

姓名 丁致宇

学号 202331060205

专业 数据科学与大数据技术

电话 18765788600

2026 年 6 月

摘 要

本报告围绕《金融经济数据挖掘》实验指导书中金融非结构化数据处理与市场情绪建模任务展开。实验一完成金融新闻文本读取、清洗、去重、交易日过滤和 BERT 三分类情感标注，并按每 5 个沪深 300 交易日聚合生成周度情绪指标；实验二基于周度正负情绪比例构造情绪偏离度、观点分歧度和综合羊群效应指数；实验三将羊群效应指标与沪深 300 周收益率对齐，构造无未来信息泄露的滞后特征和滚动统计特征，使用 LightGBM 完成非线性时序预测、特征重要性分析、SHAP 解释和双向传导验证。

实验共处理原始金融新闻 60000 条，筛选、去空、去重并剔除非交易日新闻后，进入情绪标注的样本为 30623 条，样本交易日区间为 2014-10-08 至 2015-10-21。最终生成 51 行周度情绪指标、51 行羊群效应指标、50 行市场对齐数据和 45 行机器学习特征数据。正向模型（羊群效应预测收益率）测试集 $R^2 = -0.2759$ ，说明样本外预测效果仍弱于均值基准；反向模型（收益率预测羊群效应）测试集 $R^2 = 0.0221$ ，IC 为 0.7333，方向命中率为 77.78%，显示收益变化对后续舆情羊群强度具有更强的引导信号。总体而言，本实验完成了从非结构化金融文本到结构化情绪因子、羊群效应指标和非线性预测建模的完整工程链路，同时也表明在当前样本规模下，市场收益对情绪的反馈强于情绪对收益的先行预测。

关键词：金融文本挖掘；BERT；市场情绪；羊群效应；LightGBM；SHAP；金融反身性

目录

1	实验要求理解与总体设计	4
1.1	数据与输出口径	4
1.2	程序结构	5
2	实验一：金融新闻文本预处理与周度情绪量化	6
2.1	数据清洗流程	6
2.2	BERT 情感标注	6
2.3	周度情绪指标	6
2.4	数据库存储结果	7
3	实验二：羊群效应指标构建	8
3.1	指标构建逻辑	8
3.2	羊群效应指标结果	9
3.3	数据质量分析	10
4	实验三：LightGBM 非线性预测与双向传导验证	10
4.1	数据对齐与特征工程	10
4.2	正向预测模型	10
4.3	LightGBM 特征重要性与 SHAP 分析	12
4.4	双向传导验证	13
5	AI 代码审核与修复记录	14
6	运行结果与复现说明	15
7	实验结论	17
A	完整代码附录	18
A.1	config.py	18

A.2	sentiment.py	18
A.3	data.py	20
A.4	analysis.py	23
A.5	plots.py	27
A.6	main.py	29

1 实验要求理解与总体设计

实验指导书以“金融非结构化数据预处理”为起点，但三个实验之间存在明确的数据依赖关系：实验一输出周度情绪指标，实验二在此基础上构建羊群效应指数，实验三再将羊群效应指数与沪深 300 指数收益率对齐并完成预测建模。因此，本报告采用一个完整实验链路统一呈现，同时在正文中分别对应实验一、实验二和实验三的要求。

本次实验的核心目标包括三点。第一，将金融新闻文本这类非结构化数据转化为可建模的结构化周度情绪指标；第二，依据情绪偏离和观点一致性构造羊群效应指数；第三，使用时序机器学习方法检验羊群效应与市场收益之间的预测关系和双向传导关系。为保证结果可复现，全部代码集中在 `experiment1/` 目录，原始数据位于 `data/experiment1/`，运行结果统一输出至 `outputs/experiment1/`。

1.1 数据与输出口径

实验使用两类老师配套数据：一是金融新闻文本数据，字段包括新闻编号、标题、正文、分词内容和发布时间；二是沪深 300 日度价格指数数据，用于构造交易日历和计算周收益率。新闻筛选区间设置为 2014-10-01 至 2015-10-31。周度统计单位不是普通自然周，而是按沪深 300 交易日历每 5 个交易日形成一组，从而避免周末和节假日造成的非交易日错位。

表 1 数据处理审计摘要

项目	数值
原始新闻行数	60000
日期范围与正文非空过滤后行数	36181
正文重复删除行数	1705
剔除非交易日或交易日历外新闻行数	3853
最终进入情绪标注的交易日新闻行数	30623
样本交易日范围	2014-10-08 至 2015-10-21
情绪标注方法	BERT 本地推理
周度情绪表行数	51
建模对齐数据行数	50
特征工程有效样本行数	45

1.2 程序结构

实验代码按职责拆分为多个模块。config.py 保存路径和参数，data.py 负责数据读取、清洗、交易日历构造、周度聚合和 SQLite 入库，sentiment.py 负责 BERT 情绪标注和词典回退方案，analysis.py 负责羊群效应构建、特征工程、LightGBM 建模和指标计算，plots.py 负责可视化，main.py 串联完整流程。完整源码见附录。

这种拆分方式可以减少单一脚本过长造成的维护困难，也便于逐步检查每个实验环节是否满足要求。运行入口为：

```
python -m experiment1.main
```

2 实验一：金融新闻文本预处理与周度情绪量化

2.1 数据清洗流程

实验一首先读取新闻数据 `.xls`，将发布时间字段转换为标准时间格式，然后按实验要求筛选 2014 年 10 月至 2015 年 10 月的新闻。清洗过程包括三步：删除发布时间或正文为空的样本，按正文内容去除重复新闻，使用沪深 300 交易日历剔除非交易日新闻。这样处理的原因是：空文本无法做情绪推理，重复新闻会放大某些情绪信号，非交易日新闻则难以与当日市场价格严格对齐。

清洗后，新闻按照交易日历中的 `week_id` 聚合。每个 `week_id` 对应连续 5 个沪深 300 交易日，最终周度日期采用该组最后一个交易日作为 `week` 字段。该口径与后续沪深 300 周收益率计算保持一致。

2.2 BERT 情感标注

情绪标注使用本地 HuggingFace 模型 `yiyanhokust/finbert-tone-chinese`。模型标签映射为 `Neutral` $\rightarrow$ 0、`Positive` $\rightarrow$ 1、`Negative` $\rightarrow$ -1，与本实验的正面、中性、负面三分类要求一致。程序优先使用 BERT 批量推理；当本地模型不可用时，才回退到金融情绪词典打分器。当前运行结果显示实际使用方法为 BERT。

将标题和正文拼接后输入模型，可以同时利用标题中的事件摘要和正文中的细节信息。推理时设置最大长度为 256，以控制长文本对显存和推理速度的影响。BERT 标注完成后，程序保留前 1000 条样本作为 `sentiment_labeled_sample.csv` 和数据库样例表，便于检查标注结果。

2.3 周度情绪指标

实验一要求输出 `WeekPositive`、`WeekNeutral`、`WeekNegative` 和周情绪占比 P_t 。本实验严格按指导书公式计算：

$$P_t = \frac{WeekPositive_t}{WeekPositive_t + WeekNegative_t}. \quad (1)$$

中性新闻参与 WeekNeutral 统计，但不进入 P_t 的分母。这样处理能够突出正负方向性情绪，而不是让大量中性公告稀释乐观/悲观比例。

周度情绪指标表（前8周）

week	WeekPositive	WeekNeutral	WeekNegative	NewsCount	P_t
2014-10-13	133	299	24	456	0.8471
2014-10-20	153	416	53	622	0.7427
2014-10-27	159	363	58	580	0.7327
2014-11-03	203	478	77	758	0.7250
2014-11-10	148	484	36	668	0.8043
2014-11-17	116	447	25	588	0.8227
2014-11-24	147	479	14	640	0.9130
2014-12-01	131	473	16	620	0.8912

图 1 周度情绪指标表截图

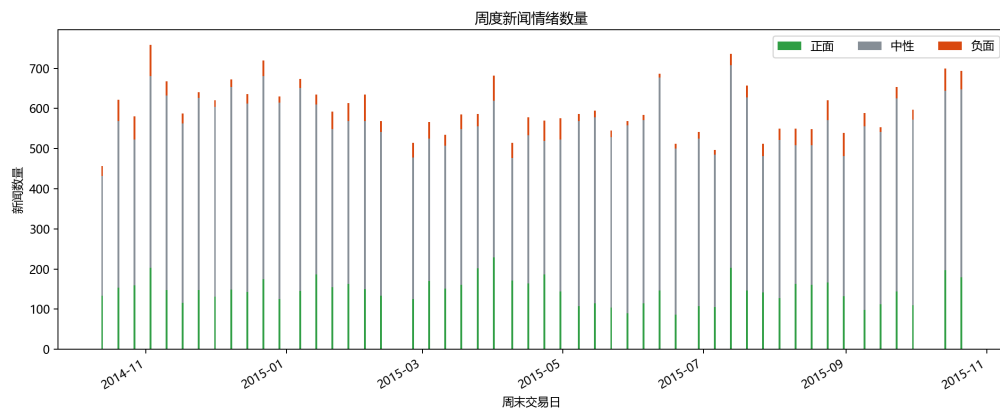


图 2 周度新闻情绪数量堆叠图

从图 1 可见，周度情绪表包含实验要求的核心字段，并额外保留周起止日期、交易周编号和新闻总量，便于审计分组口径。图 2 显示，中性新闻数量整体较高，正面新闻数量通常高于负面新闻，说明新闻文本中存在明显的中性公告和偏正面信息披露特征。

2.4 数据库存储结果

实验一要求将周度情绪表存入数据库并导出 CSV。本实验使用 SQLite 数据库 outputs/experiment1/experiment1.db 保存关键结果，包括情绪

样例、周度情绪、羊群指标、沪深 300 周收益和建模对齐数据。

SQLite 数据库存储结果

table_name	row_count
weekly_sentiment	51
weekly_herd_index	51
hs300_weekly_retur...	588
modeling_dataset	50
sentiment_labeled_...	1000

图 3 SQLite 数据库存储结果截图

图 3 表明，周度情绪表共有 51 行，与导出的 CSV 行数一致；羊群指标表同样为 51 行；建模对齐表在剔除首期历史基准缺失和收益率缺失后保留 50 行。

3 实验二：羊群效应指标构建

3.1 指标构建逻辑

实验二的任务是基于周度情绪指标构造羊群效应指数。单纯的乐观比例 P_t 只能说明当周正面情绪占比，并不能直接表示羊群效应。真正的羊群行为应同时满足两个条件：其一，当周情绪相对历史水平出现明显偏离；其二，市场观点呈现单边一致，而不是多空均衡。

因此，本实验构造三个指标。首先，用过去 4 周 P_t 均值作为正常情绪基准：

$$E(P_t) = \frac{1}{4} \sum_{i=1}^4 P_{t-i}. \quad (2)$$

其次，构造情绪偏离度：

$$H1_t = P_t - E(P_t). \quad (3)$$

再次，构造观点分歧度：

$$H2_t = 1 - \frac{|WeekPositive_t - WeekNegative_t|}{WeekPositive_t + WeekNegative_t}. \quad (4)$$

需要注意的是，按该定义， $H2_t$ 越小表示正负观点越单边，羊群一致性越强；

$H2_t$ 越大表示正负观点越均衡，分歧越强。因此，最终综合指标采用情绪偏离强度与单边一致性相乘：

$$H3_t = Norm(|H1_t|) \times (1 - Norm(H2_t)). \quad (5)$$

其中 $Norm(\cdot)$ 表示 Min-Max 归一化。该实现与“情绪反常且观点一边倒时羊群效应更强”的金融逻辑一致。

3.2 羊群效应指标结果

羊群效应指标表（前8周）

week	P_t	E_P_t	$H1_t$	$H2_t$	$H3_t$
2014-10-13	0.8471	nan	nan	0.3057	nan
2014-10-20	0.7427	0.8471	-0.1044	0.5146	0.1392
2014-10-27	0.7327	0.7949	-0.0622	0.5346	0.0648
2014-11-03	0.7250	0.7742	-0.0492	0.5500	0.0405
2014-11-10	0.8043	0.7619	0.0425	0.3913	0.1246
2014-11-17	0.8227	0.7512	0.0715	0.3546	0.2506
2014-11-24	0.9130	0.7712	0.1419	0.1739	0.8613
2014-12-01	0.8912	0.8163	0.0749	0.2177	0.4032

图 4 羊群效应指标表截图

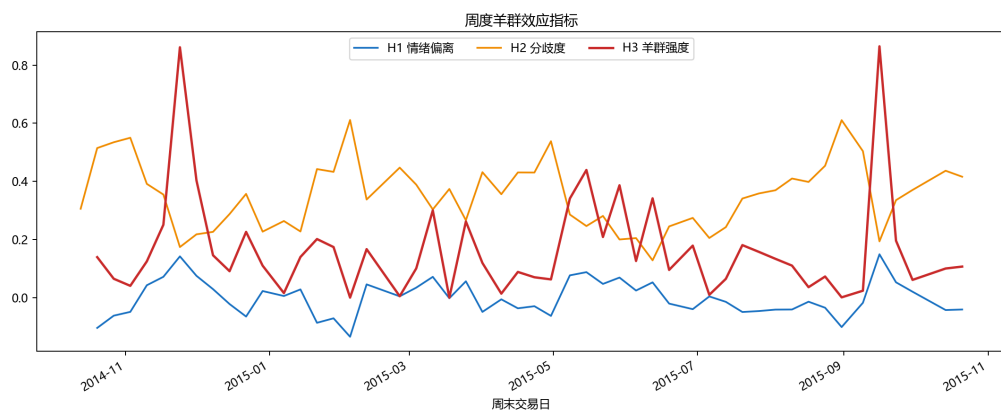


图 5 周度羊群效应指标时序图

图 4 展示了 P_t 、 $E(P_t)$ 、 $H1_t$ 、 $H2_t$ 和 $H3_t$ 的样例结果。首期由于不存在历史 4 周基准， $E(P_t)$ 和 $H1_t$ 为空，后续建模时自动剔除。图 5 显示， $H3_t$ 并非简

单跟随 P_t 上下波动，而是在情绪偏离和单边一致性同时较强时才出现高值，这符合羊群效应指数的设计目标。

3.3 数据质量分析

羊群指标的数据质量主要体现在三方面。第一，所有指标均基于交易日过滤后的新闻样本，避免了非交易日文本与价格序列错位。第二， P_t 的分母只使用正负情绪文本，避免中性公告数量过大造成方向性情绪被稀释。第三， $H1_t$ 使用过去 4 周均值作为基准，严格使用历史信息，避免使用未来情绪数据。后续机器学习特征中的滞后项和滚动统计也已先进行 $\text{shift}(1)$ ，保证预测当周收益时只使用过去已知信息。

4 实验三：LightGBM 非线性预测与双向传导验证

4.1 数据对齐与特征工程

实验三将实验二输出的周度羊群效应指标与沪深 300 周收益率按 `week_id` 和周末交易日内连接对齐。沪深 300 周收益率使用每 5 个交易日分组后的最后收盘价计算：

$$Ret_t = \frac{Close_t - Close_{t-1}}{Close_{t-1}}. \quad (6)$$

特征工程围绕 $H3_t$ 构造。核心滞后特征包括 $H3_lag1$ 至 $H3_lag5$ ，分别表示过去 1 至 5 周羊群效应；滚动统计特征包括过去 3 周和 5 周的均值与标准差；时间特征包括月份和季度。所有滚动特征均先对原序列进行 $\text{shift}(1)$ 再滚动计算，确保不包含目标周的同期情绪。

4.2 正向预测模型

正向模型使用过去的羊群效应特征预测当周沪深 300 收益率。数据按时间顺序划分，前 80% 作为训练集，后 20% 作为测试集，不使用随机划分。模型采用 LightGBM 回归器，参数设置为学习率 0.1、最大深度 3、叶子节点数 7、估计

器数量 200，并设置随机种子 42。

表 2 正向模型测试集指标

指标	数值
MSE	0.006118
MAE	0.053541
R^2	-0.2759
IC (Spearman 秩相关)	0.2500
方向命中率	44.44%
测试集样本数	9

表 2 表明，正向模型测试集 R^2 为负，说明在当前样本量下，模型样本外预测误差仍高于简单均值基准。IC 为 0.2500，说明预测排序存在一定弱信号，但方向命中率只有 44.44%，不能认为羊群效应对当周收益具有稳定预测能力。

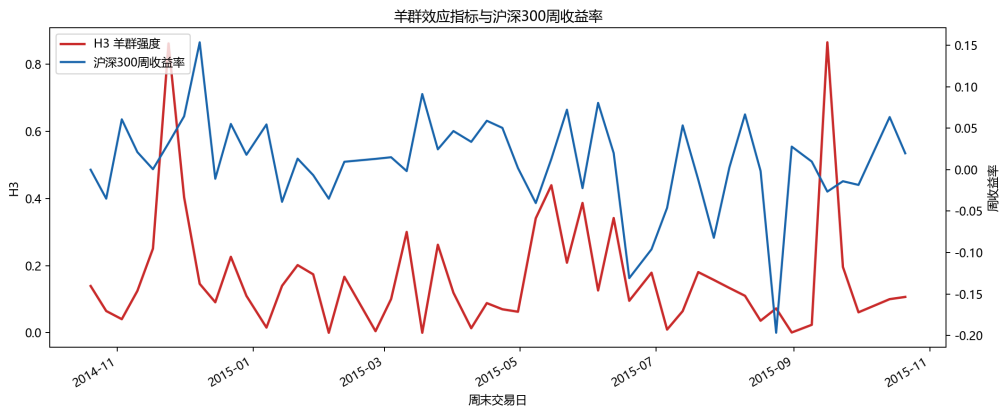


图 6 羊群效应指标与沪深 300 周收益率对比

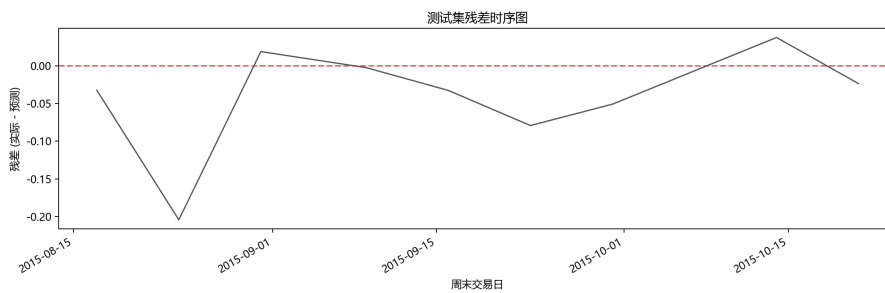


图 7 正向模型测试集残差时序图

图 6 显示，羊群效应与收益率在若干时期存在共同波动，但整体关系并不稳定。图 7 显示，测试集残差存在明显波动，说明单独依靠情绪羊群特征难以稳定解释短期指数收益。

4.3 LightGBM 特征重要性与 SHAP 分析

为了区分模型内部 gain 重要性和 SHAP 解释，本实验分别输出 `lgbm_feature_importance.csv` 和 `shap_importance.csv`。LightGBM gain 重要性衡量特征在树分裂中带来的损失下降，SHAP 则从样本层面解释每个特征对预测值的边际贡献。

LightGBM Gain 特征重要性 Top 8

特征	Gain重要性
H3_lag5	0.0951
month	0.0462
H3_lag1	0.0345
H3_lag2	0.0139
H3_rol_mean_5	0.0129
H3_rol_mean_3	0.0118
H3_lag4	0.0086
H3_rol_std_5	0.0065

图 8 LightGBM Gain 特征重要性表截图

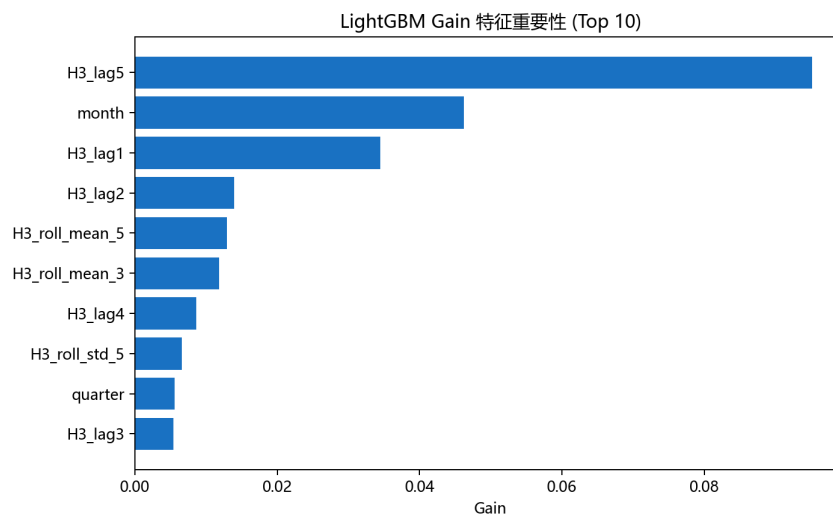


图 9 LightGBM Gain 特征重要性图

从图 8 和图 9 可以看出，最重要的羊群效应特征为 $H3_lag5$ ，说明在当前样本中，5 周前的羊群效应对收益预测贡献最大。月份特征也具有较高 gain，说明样本中可能存在阶段性行情或季节性结构，但由于测试集只有 9 条，不能过度解释。

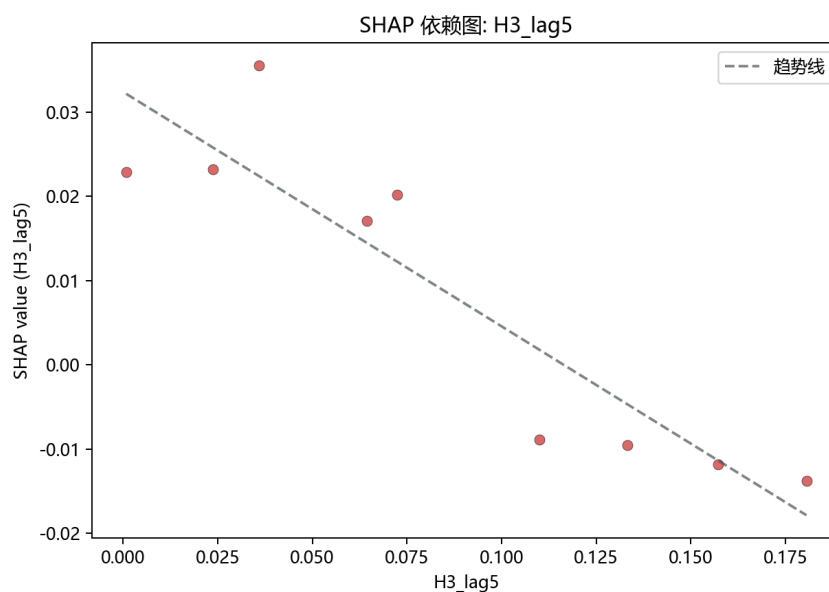


图 10 SHAP 依赖图

图 10 以 SHAP 值展示最重要特征对收益预测的影响。该图的意义不是证明线性显著关系，而是说明 LightGBM 模型如何在测试样本中利用 $H3_lag5$ 调整预测值。由于样本量较小，SHAP 图更适合作为模型解释证据，而不能单独作为强统计结论。

4.4 双向传导验证

为了验证金融反身性，本实验构建两个方向的模型：正向模型使用滞后羊群效应预测收益率，反向模型使用滞后收益率预测羊群效应。反向模型的滚动收益特征同样先进行 $\text{shift}(1)$ ，避免当周收益泄露。反向方向命中率不再使用 $H3_t$ 的正负号，因为 $H3_t$ 本身非负，而是比较预测 $H3_t$ 相对上一期 $H3_{t-1}$ 的涨跌方向是否正确。

双向建模结果对比

建模方向	MSE	MAE	R2	IC	方向命中率	n_test	最优滞后
H3→return (forward)	0.0061	0.0535	-0.2759	0.2500	44.44%	9	5
return→H3 (backward)	0.0631	0.1364	0.0221	0.7333	77.78%	9	2

图 11 双向建模结果表截图

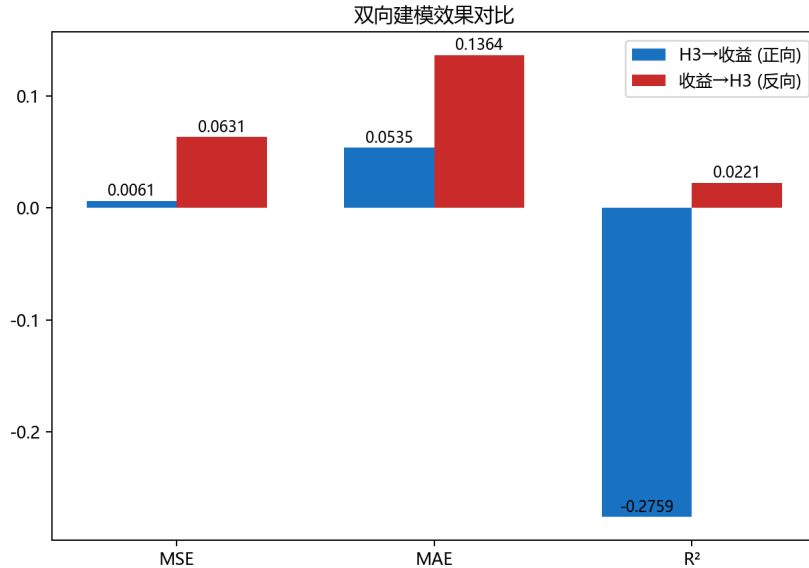


图 12 双向建模效果对比图

图 11 和图 12 表明，反向模型测试集 $R^2 = 0.0221$ ，高于正向模型的 $R^2 = -0.2759$ ；反向 IC 为 0.7333，也高于正向 IC 0.2500。虽然反向模型解释力仍然较弱，但相对正向模型更稳定，说明当前数据中“市场收益变化影响后续情绪羊群强度”的反馈路径强于“情绪羊群强度预测后续收益”的先行路径。

5 AI 代码审核与修复记录

实验指导要求提交 AI 代码审核表。本节记录本次 AI 辅助审查发现的问题、修复方式和验证结果。审核重点不是形式化罗列，而是确认代码是否满足金融时间序列任务中的关键约束。

表3 AI 代码审核与修复记录

审核项	发现问题	修复或确认结果	状态
情绪公式	检查 P_t 分母是否应包含中性新闻。	对照实验公式后确认 $P_t = Positive / (Positive + Negative)$, 代码实现正确。	通过
羊群指标	$H2_t$ 越小表示越一致, 若直接乘 $Norm(H2_t)$ 会与解释矛盾。	代码采用 $Norm(H1_t) \times (1 - Norm(H2_t))$, 并在代码注释中明确 $H2_t$ 为分歧度。	通过
时序泄露	原滚动均值和滚动标准差包含当周 $H3_t$ 或当周收益。	已改为先 <code>shift(1)</code> 再滚动, 确保特征只使用目标周之前的信息。	已修复
特征重要性	原图把 SHAP 平均绝对值误标为 LightGBM Gain。	新增 LightGBM gain 明细输出, 图表改为真正的 LightGBM gain; SHAP 单独输出。	已修复
反向命中率	$H3_t$ 非负, 直接用符号计算方向准确率会虚高。	改为相对上一期 $H3$ 的涨跌方向命中率, 结果为 77.78%。	已修复
可复现性	需要验证主流程能从原始数据重新生成结果。	已复跑 <code>python -m experiment1.main</code> , BERT 标注 30623 条并生成全部输出。	通过
数据库存储	需要确认 CSV 与 SQLite 结果存在。	SQLite 中周度情绪、羊群指标、建模数据集等核心结果表均存在。	通过

6 运行结果与复现说明

本实验在 Windows 环境下运行, Python 版本为 3.11.5, 主要依赖包括 pandas、numpy、matplotlib、scikit-learn、lightgbm、scipy、xlrd、transformers、torch 和 shap。依赖列表保存在 requirements.txt。复现实验时, 在仓库根目录执行:

```
python -m experiment1.main
```

程序会自动读取 data/experiment1/raw/ 中的新闻数据和沪深 300 日度价格数据, 生成周度情绪、羊群指标、建模数据、CSV、SQLite 数据库、图表和摘要 JSON。核心输出文件见表 4。

表 4 核心输出文件清单

文件	说明
weekly_sentiment.csv	实验一周度情绪指标表。
weekly_herd_index.csv	实验二羊群效应指标表。
hs300_weekly_return.csv	沪深 300 按 5 个交易日聚合后的周收益表。
modeling_dataset.csv	羊群效应与沪深 300 收益率对齐后的建模数据。
feature_engineering.csv	滞后特征、滚动统计特征和时间特征数据集。
lgbm_forward_results.csv	正向 LightGBM 测试集预测、真实收益和残差。
lgbm_feature_importance.csv	LightGBM gain 特征重要性。
shap_importance.csv	SHAP 平均绝对贡献度排序。
bidirectional_comparison.csv	正向与反向建模指标对比。
experiment1.db	SQLite 数据库，保存核心结果表。
summary.json	全流程审计、指标、图表和解释摘要。

7 实验结论

第一，实验一已经完成从非结构化金融新闻到结构化周度情绪指标的转换。经过清洗、去重、交易日过滤和 BERT 情绪标注后，30623 条新闻进入周度聚合，生成 51 行周度情绪数据，并同时导出 CSV 和写入 SQLite 数据库。

第二，实验二构造的羊群效应指数能够同时反映情绪偏离和单边一致性。单纯的乐观比例 P_t 不能代表羊群效应，必须结合历史基准和观点分歧度。 $H3_t$ 的高值对应“情绪异常且观点一边倒”的时期，符合金融羊群效应的指标逻辑。

第三，实验三表明，在当前小样本条件下，羊群效应对沪深 300 周收益率的正向预测能力有限。正向模型 R^2 为负，说明其预测误差仍大于均值基准；虽然 $H3_lag5$ 在 gain 和 SHAP 中均排名靠前，但这只能说明模型使用了该特征，不能证明其具有稳定强预测能力。

第四，反向模型的表现相对更好，说明市场收益对后续情绪和羊群强度的影响更明显。这与金融反身性理论一致：市场涨跌会改变新闻叙事和投资者情绪，进而影响下一阶段的群体行为。当前数据更支持“收益驱动舆情”的反馈路径，而不是“舆情稳定预测收益”的单向路径。

第五，实验仍存在样本规模较小、测试集只有 9 周、情绪模型未针对本数据集重新微调等限制。后续若要增强结论可靠性，应扩展新闻和行情时间跨度，加入成交量、波动率、宏观变量和行业变量，并采用滚动窗口回测检验模型稳定性。

A 完整代码附录

本附录列出实验一完整核心代码。代码以仓库中的实际文件为准，路径为 experiment1/。

A.1 config.py

```
"""Shared paths and experiment parameters."""

from __future__ import annotations
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
PACKAGE_DIR = ROOT / "experiment1"
DATA_DIR = ROOT / "data" / "experiment1"
RAW_DIR = DATA_DIR / "raw"
OUTPUT_DIR = ROOT / "outputs" / "experiment1"
REPORT_DIR = ROOT / "report"

NEWS_FILE = RAW_DIR / "新闻数据.xls"
HS300_FILE = RAW_DIR / "沪深 300 日价格指数.xls"
DB_FILE = OUTPUT_DIR / "experiment1.db"

# BERT model stored in parent directory to keep the repo lean
MODEL_DIR = ROOT.parent / "models" / "yiyanghkust-finbert-tone-chinese"

START_DATE = "2014-10-01"
END_DATE = "2015-10-31"
TRADING_DAYS_PER_WEEK = 5
ROLLING_BASELINE_WEEKS = 4
BERT_BATCH_SIZE = 256

# Experiment 3: LightGBM parameters
MAX_LAG = 5
TRAIN_RATIO = 0.8
LGBM_PARAMS = {
    "objective": "regression",
    "metric": "mse",
    "learning_rate": 0.1,
    "num_leaves": 7,
    "max_depth": 3,
    "n_estimators": 200,
    "min_child_samples": 2,
    "reg_alpha": 0.1,
    "reg_lambda": 0.1,
    "random_state": 42,
    "verbose": -1,
    "n_jobs": -1,
}

STUDENT_ID = "202331060205"
STUDENT_NAME = "丁致宇"
```

A.2 sentiment.py

```
"""Sentiment labeling for Chinese financial news.

Default strategy: local FinBERT three-class inference via
`yiyanghkust/finbert-tone-chinese` (Positive / Neutral / Negative).
If the model files are not found, falls back to a transparent financial
lexicon scorer so the pipeline always produces reproducible results.
"""

from __future__ import annotations

import re
from collections.abc import Iterable

import numpy as np
```

```

import pandas as pd

from . import config

# -----
# BERT-based labeling (direct model inference, FP16, dynamic padding)
# -----

_bert_ctx: dict | None = None

# Model id2label: {'0': 'Neutral', '1': 'Positive', '2': 'Negative'}
_MODEL_ID2LABEL = {0: 0, 1: 1, 2: -1} # Neutral→0, Positive→1, Negative→-1

def _get_bert_ctx() -> dict | None:
    """Lazy-load tokenizer + model on GPU with FP16."""
    global _bert_ctx
    if _bert_ctx is not None:
        return _bert_ctx
    if not config.MODEL_DIR.exists():
        return None
    import torch
    from transformers import AutoModelForSequenceClassification, AutoTokenizer

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    tokenizer = AutoTokenizer.from_pretrained(str(config.MODEL_DIR),
        ↪ local_files_only=True)
    model = AutoModelForSequenceClassification.from_pretrained(
        str(config.MODEL_DIR), local_files_only=True, torch_dtype=torch.float16
    ).to(device)
    model.eval()
    _bert_ctx = {"tokenizer": tokenizer, "model": model, "device": device}
    return _bert_ctx

def bert_label_batch(texts: list[str], batch_size: int = 256) -> list[int]:
    """Run BERT on texts using direct model inference (GPU FP16, fast)."""
    import torch

    ctx = _get_bert_ctx()
    if ctx is None:
        raise RuntimeError("BERT model not available")
    tok = ctx["tokenizer"]
    model = ctx["model"]
    device = ctx["device"]

    clean = [str(t)[:256] if pd.notna(t) else "" for t in texts]
    total = len(clean)
    labels: list[int] = []

    with torch.no_grad():
        for start in range(0, total, batch_size):
            batch = clean[start : start + batch_size]
            enc = tok(batch, padding=True, truncation=True, max_length=256,
                ↪ return_tensors="pt")
            enc = {k: v.to(device) for k, v in enc.items()}
            logits = model(**enc).logits
            preds = logits.argmax(dim=-1).cpu().tolist()
            labels.extend(_MODEL_ID2LABEL[p] for p in preds)
            done = min(start + batch_size, total)
            if done % (batch_size * 4) == 0 or done == total:
                print(f" BERT labeling: {done}/{total} ({100*done//total}%)",
                    ↪ flush=True)
    return labels

def bert_label_text(text: str) -> int:
    """Run BERT on a single text and return {-1, 0, 1}."""
    return bert_label_batch([text], batch_size=1)[0]

# -----
# Lexicon-based fallback
# -----

POSITIVE_WORDS = {
    "上涨", "涨停", "大涨", "走高", "反弹", "拉升", "攀升", "上扬", "领涨", "飙升",
    "突破", "创新高", "增长", "增加", "提升", "改善", "回暖", "复苏", "盈利", "利润",
    "利好", "看好", "乐观", "强劲", "稳健", "受益", "增持", "买入", "推荐", "超预期",

```

```

    "扩张", "订单", "中标", "签约", "并购", "重组", "注入", "分红", "高送转", "回购",
    "政策支持", "获批", "通过", "增长率", "净利", "同比增", "业绩预增", "扭亏",
    "提振", "活跃", "放量", "资金流入", "牛市", "景气", "领先", "升逾",
}

NEGATIVE_WORDS = {
    "下跌", "跌停", "大跌", "走低", "回落", "跳水", "杀跌", "领跌", "破位", "下滑",
    "下降", "减少", "亏损", "亏", "利空", "悲观", "承压", "风险", "警惕", "减持",
    "卖出", "下调", "低于预期", "违规", "调查", "处罚", "诉讼", "债务", "违约",
    "退市", "停牌", "质押", "爆仓", "裁员", "业绩预减", "业绩下滑", "流出",
    "萎缩", "疲软", "砸盘", "出逃", "套现", "跌逾", "低迷", "恶化", "不确定",
    "压力", "受挫", "放缓", "亏损扩大", "黑天鹅", "危机", "熊市",
}

NEGATIONS = {"不", "未", "无", "没有", "并未", "难以", "不能", "不是"}

def _tokens(segmented: str | None, title: str | None, content: str | None) ->
    ↪ list[str]:
    parts: list[str] = []
    if segmented:
        parts.extend(str(segmented).split("|"))
    if title:
        parts.extend(re.findall(r"[\u4e00-\u9fffA-Za-z0-9]+", str(title)))
    if content:
        parts.extend(re.findall(r"[\u4e00-\u9fffA-Za-z0-9]+", str(content)))
    return [p.strip() for p in parts if p and p.strip()]

def _contains_negation(tokens: list[str], index: int) -> bool:
    start = max(0, index - 3)
    return any(tokens[i] in NEGATIONS for i in range(start, index))

def score_text(segmented: str | None, title: str | None, content: str | None)
    ↪ -> int:
    tokens = _tokens(segmented, title, content)
    score = 0
    for i, token in enumerate(tokens):
        weight = 2 if i < 30 else 1
        pos_hit = any(word in token for word in POSITIVE_WORDS)
        neg_hit = any(word in token for word in NEGATIVE_WORDS)
        if pos_hit and not neg_hit:
            score += weight if _contains_negation(tokens, i) else weight
        elif neg_hit and not pos_hit:
            score += weight if _contains_negation(tokens, i) else -weight
    return score

def label_from_score(score: int, neutral_band: int = 1) -> int:
    if score > neutral_band:
        return 1
    if score < -neutral_band:
        return -1
    return 0

def label_name(label: int) -> str:
    return {1: "positive", 0: "neutral", -1: "negative"}[int(label)]

def summarize_labels(labels: Iterable[int]) -> dict[str, int]:
    values = list(labels)
    return {
        "positive": sum(v == 1 for v in values),
        "neutral": sum(v == 0 for v in values),
        "negative": sum(v == -1 for v in values),
    }

```

A.3 data.py

```

"""Data loading, cleaning, and persistence helpers."""

from __future__ import annotations

import sqlite3

```

```

from pathlib import Path

import pandas as pd

from . import config
from .sentiment import bert_label_batch, label_from_score, label_name,
↳ score_text

def load_news() -> pd.DataFrame:
    df = pd.read_excel(config.NEWS_FILE)
    df.columns = [str(c).strip() for c in df.columns]
    df["date_time"] = pd.to_datetime(df["Time"], errors="coerce")
    return df

def load_hs300_daily() -> pd.DataFrame:
    raw = pd.read_csv(config.HS300_FILE, sep="\t", encoding="gbk", skiprows=2)
    raw.columns = [str(c).strip() for c in raw.columns]
    keep = raw.rename(
        columns={"时间": "date", "开盘": "open", "最高": "high", "最低": "low",
↳ "收盘": "close", "成交量": "volume"}
    )
    keep = keep[["date", "open", "high", "low", "close", "volume"]].copy()
    keep["date"] = pd.to_datetime(keep["date"], errors="coerce")
    for col in ["open", "high", "low", "close", "volume"]:
        keep[col] = pd.to_numeric(keep[col], errors="coerce")
    keep = keep.dropna(subset=["date",
↳ "close"]).sort_values("date").reset_index(drop=True)
    return keep

def make_trading_calendar(hs300: pd.DataFrame) -> pd.DataFrame:
    cal = hs300[["date"]].drop_duplicates().sort_values("date").reset_index(drop=
↳ p=True)
    cal["trade_index"] = range(len(cal))
    cal["week_id"] = cal["trade_index"] // config.TRADING_DAYS_PER_WEEK
    week_bounds = cal.groupby("week_id")["date"].agg(week_start="min",
↳ week_end="max").reset_index()
    return cal.merge(week_bounds, on="week_id", how="left")

def clean_and_label_news(news: pd.DataFrame, calendar: pd.DataFrame) ->
↳ tuple[pd.DataFrame, dict]:
    start = pd.Timestamp(config.START_DATE)
    end = pd.Timestamp(config.END_DATE) + pd.Timedelta(days=1) -
↳ pd.Timedelta(seconds=1)
    before_rows = len(news)
    df = news.dropna(subset=["date_time", "Content"]).copy()
    df = df[(df["date_time"] >= start) & (df["date_time"] <= end)].copy()
    df["trade_date"] = df["date_time"].dt.normalize()
    before_dedup = len(df)
    df = df.drop_duplicates(subset=["Content"]).copy()
    after_dedup = len(df)
    cal = calendar[["date", "week_id", "week_start",
↳ "week_end"]].rename(columns={"date": "trade_date"})
    df = df.merge(cal, on="trade_date", how="inner")

    # Prefer BERT batch labeling; fall back to lexicon if model unavailable
    use_bert = config.MODEL_DIR.exists()
    labeling_method = "lexicon"
    if use_bert:
        try:
            texts = (df["Title"].fillna("") + " " +
↳ df["Content"].fillna("")).tolist()
            df["label"] = bert_label_batch(texts,
↳ batch_size=config.BERT_BATCH_SIZE)
            labeling_method = "bert"
        except Exception:
            use_bert = False

    if not use_bert:
        df["sentiment_score"] = [
            score_text(seg, title, content)
            for seg, title, content in zip(df.get("SegContent"),
↳ df.get("Title"), df.get("Content"))
        ]
        df["label"] = df["sentiment_score"].map(label_from_score)

```

```

df["label_name"] = df["label"].map(label_name)
df = df.sort_values(["date_time", "NewsID"]).reset_index(drop=True)
audit = {
    "raw_news_rows": before_rows,
    "rows after date and non null filter": before_dedup,
    "duplicate_content_removed": before_dedup - after_dedup,
    "labeled trading_day_rows": len(df),
    "non_trading_or_out_of_calendar_removed": after_dedup - len(df),
    "date_start": str(df["trade_date"].min().date()) if not df.empty else
    ↪ " ",
    "date_end": str(df["trade_date"].max().date()) if not df.empty else " ",
    "labeling_method": labeling_method,
}
return df, audit

def build_weekly_sentiment(labeled: pd.DataFrame) -> pd.DataFrame:
    weekly = (
        labeled.groupby(["week_id", "week_start", "week_end"])["label"]
        .agg(
            WeekPositive=lambda x: int((x == 1).sum()),
            WeekNeutral=lambda x: int((x == 0).sum()),
            WeekNegative=lambda x: int((x == -1).sum()),
            NewsCount="count",
        )
        .reset_index()
    )
    denom = weekly["WeekPositive"] + weekly["WeekNegative"]
    weekly["P_t"] = (weekly["WeekPositive"] / denom).where(denom.ne(0), 0.5)
    weekly["week"] = weekly["week_end"]
    columns = ["week", "week_start", "week_end", "week_id", "WeekPositive",
    ↪ "WeekNeutral", "WeekNegative", "NewsCount", "P_t"]
    return weekly[columns].sort_values("week").reset_index(drop=True)

def build_hs300_weekly(hs300: pd.DataFrame, calendar: pd.DataFrame) ->
    ↪ pd.DataFrame:
    df = hs300.merge(calendar[["date", "week_id", "week_start", "week_end"]],
    ↪ on="date", how="inner")
    weekly = (
        df.groupby(["week_id", "week_start", "week_end"])
        .agg(open=("open", "first"), close=("close", "last"), high=("high",
    ↪ "max"), low=("low", "min"), volume=("volume", "sum"))
        .reset_index()
    )
    weekly["return"] = weekly["close"].pct_change()
    weekly["week"] = weekly["week_end"]
    return weekly[["week", "week_start", "week_end", "week_id", "open", "high",
    ↪ "low", "close", "volume", "return"]]

def save_outputs(
    labeled: pd.DataFrame,
    weekly_sentiment: pd.DataFrame,
    weekly_herd: pd.DataFrame,
    hs300_weekly: pd.DataFrame,
    modeling: pd.DataFrame,
) -> None:
    config.OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    sample_cols = [c for c in ["NewsID", "Title", "date_time", "trade_date",
    ↪ "week_id", "sentiment_score", "label", "label_name"] if c in
    ↪ labeled.columns]
    labeled_sample = labeled[sample_cols].head(1000)
    labeled_sample.to_csv(config.OUTPUT_DIR / "sentiment_labeled_sample.csv",
    ↪ index=False, encoding="utf-8-sig")
    weekly_sentiment.to_csv(config.OUTPUT_DIR / "weekly_sentiment.csv",
    ↪ index=False, encoding="utf-8-sig")
    weekly_herd.to_csv(config.OUTPUT_DIR / "weekly_herd_index.csv",
    ↪ index=False, encoding="utf-8-sig")
    hs300_weekly.to_csv(config.OUTPUT_DIR / "hs300_weekly_return.csv",
    ↪ index=False, encoding="utf-8-sig")
    modeling.to_csv(config.OUTPUT_DIR / "modeling_dataset.csv", index=False,
    ↪ encoding="utf-8-sig")
    with sqlite3.connect(config.DB_FILE) as conn:
        labeled_sample.to_sql("sentiment_labeled_sample", conn,
    ↪ if_exists="replace", index=False)
        weekly_sentiment.to_sql("weekly_sentiment", conn, if_exists="replace",
    ↪ index=False)
        weekly_herd.to_sql("weekly_herd_index", conn, if_exists="replace",
    ↪ index=False)

```

```
hs300_weekly.to_sql("hs300_weekly_return", conn, if_exists="replace",
    ↪ index=False)
modeling.to_sql("modeling_dataset", conn, if_exists="replace",
    ↪ index=False)

def write_data_description(audit: dict, path: Path | None = None) -> None:
    path = path or (config.DATA_DIR / " 数据说明.md")
    path.parent.mkdir(parents=True, exist_ok=True)
    text = f"""# 实验一配套数据说明

## 原始资料

- `original/金融数据挖掘实验指导书 20260528.docx`: 老师发布的实验指导书原件。
- `original/实验配套数据.zip`: 老师发布的实验配套数据压缩包。
- `raw/新闻数据.xls`: 金融新闻文本数据, 字段包括 `ID`、`NewsID`、`SegContent`、`Title`、
    ↪ `Content`、`Time`。
- `raw/沪深300日价格指数.xls`: 沪深300日价格数据。该文件实际为 GBK 编码的制表符文本, 扩展名为
    ↪ `.xls`。

## 本次处理口径

- 新闻筛选区间: {config.START_DATE} 至 {config.END_DATE}。
- 周度统计单位: 严格按沪深300交易日历每 {config.TRADING_DAYS_PER_WEEK} 个交易日分为一组,
    ↪ 而不是简单日历周。
- 清洗步骤: 删除时间或正文为空的数据, 按正文去重, 剔除非交易日新闻。
- 情绪标注: 使用 `yiyanghkust/finbert-tone-chinese` 模型进行 BERT 三分类推理
    ↪ (正面/中性/负面); 若模型文件不可用则回退到金融情绪词典打分器。

## 清洗摘要

- 原始新闻行数: {audit.get('raw_news_rows')}
- 日期与非空过滤后行数: {audit.get('rows_after_date_and_non_null_filter')}
- 最终进入交易日样本行数: {audit.get('labeled_trading_day_rows')}
- 样本日期范围: {audit.get('date_start')} 至 {audit.get('date_end')}

## 主要输出

- `outputs/experiment1/weekly_sentiment.csv`: 实验一周度情绪表。
- `outputs/experiment1/weekly_herd_index.csv`: 实验二羊群效应指标表。
- `outputs/experiment1/modeling_dataset.csv`: 实验三建模对齐数据。
- `outputs/experiment1/experiment1.db`: SQLite 数据库, 包含周度情绪、羊群指标、
    ↪ 沪深300周收益和建模数据表。
"""
    path.write_text(text, encoding="utf-8")
```

A.4 analysis.py

```
"""Herd-index construction and LightGBM market-prediction analysis."""

from __future__ import annotations

import warnings

import numpy as np
import pandas as pd
import lightgbm as lgb
import shap

from . import config

# -----
# Experiment 2: Herd index
# -----

def _minmax(series: pd.Series) -> pd.Series:
    s = series.astype(float)
    lo, hi = s.min(), s.max()
    if pd.isna(lo) or pd.isna(hi) or hi == lo:
        return pd.Series(0.0, index=s.index)
    return (s - lo) / (hi - lo)

def winsorize_series(series: pd.Series, sigma: float = 3.0) -> pd.Series:
```



```

mean, std = series.mean(), series.std(ddof=0)
if pd.isna(std) or std == 0:
    return series
return series.clip(mean - sigma * std, mean + sigma * std)

def build_herd_index(weekly_sentiment: pd.DataFrame) -> pd.DataFrame:
    df = weekly_sentiment.copy().sort_values("week").reset_index(drop=True)
    df["E_P_t"] = df["P_t"].shift(1).rolling(config.ROLLING_BASELINE_WEEKS,
        ↪ min_periods=1).mean()
    # First row has no history — leave E_P_t as NaN so H1t/H3t are NaN and
    # dropped downstream rather than biased to zero.
    df["H1t"] = df["P_t"] - df["E_P_t"]
    denom = df["WeekPositive"] + df["WeekNegative"]
    df["H2t"] = (1 - (df["WeekPositive"] - df["WeekNegative"]).abs() /
        ↪ denom).where(denom.ne(0), 1.0)
    df["H1t_winsor"] = winsorize_series(df["H1t"])
    df["H2t_winsor"] = winsorize_series(df["H2t"])
    df["H1t_norm"] = _minmax(df["H1t_winsor"].abs())
    df["H2t_norm"] = _minmax(df["H2t_winsor"])
    # H2t is a disagreement score: smaller H2t means stronger one-sided
    ↪ consensus.
    df["H3t"] = df["H1t_norm"] * (1 - df["H2t_norm"])
    columns = [
        "week", "week_start", "week_end", "week_id", "P_t", "E_P_t", "H1t",
        ↪ "H2t",
        "H1t_norm", "H2t_norm", "H3t",
        "WeekPositive", "WeekNeutral", "WeekNegative", "NewsCount",
    ]
    return df[columns]

def build_modeling_dataset(weekly_herd: pd.DataFrame, hs300_weekly:
    ↪ pd.DataFrame) -> pd.DataFrame:
    df = weekly_herd.merge(
        hs300_weekly[["week_id", "week", "close", "return"]],
        on=["week_id", "week"], how="inner",
    )
    return df.dropna(subset=["H3t",
        ↪ "return"]).sort_values("week").reset_index(drop=True)

# -----
# Experiment 3: Feature engineering + LightGBM
# -----

def build_features(df: pd.DataFrame) -> pd.DataFrame:
    """Construct lag features, rolling statistics, and time features."""
    out = df.copy()
    # Lag features for H3t
    for lag in range(1, config.MAX_LAG + 1):
        out[f"H3_lag{lag}"] = out["H3t"].shift(lag)
    # Rolling statistics use only information available before the target week.
    h3_past = out["H3t"].shift(1)
    for w in [3, 5]:
        out[f"H3_roll_mean_{w}"] = h3_past.rolling(w, min_periods=w).mean()
        out[f"H3_roll_std_{w}"] = h3_past.rolling(w, min_periods=w).std()
    # Time features (use week_end as datetime)
    week_dt = pd.to_datetime(out["week"])
    out["month"] = week_dt.dt.month
    out["quarter"] = week_dt.dt.quarter
    return out

FEATURE_COLS = (
    [f"H3_lag{i}" for i in range(1, config.MAX_LAG + 1)]
    + [f"H3_roll_mean_{w}" for w in [3, 5]]
    + [f"H3_roll_std_{w}" for w in [3, 5]]
    + ["month", "quarter"]
)

def temporal_train_test_split(df: pd.DataFrame) -> tuple[pd.DataFrame,
    ↪ pd.DataFrame]:
    """Strict temporal split: first 80% train, last 20% test."""
    n = len(df)
    split = int(n * config.TRAIN_RATIO)
    return df.iloc[:split].copy(), df.iloc[split:].copy()

```

```

def train_lgbm(
    X_train,
    y_train,
    X_test,
    y_test,
    direction_baseline=None,
) -> tuple[lgb.LGBMRegressor, dict, np.ndarray]:
    """Train LightGBM, return model, metrics dict, test predictions."""
    model = lgb.LGBMRegressor(**config.LGBM_PARAMS)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    metrics = evaluate(y_test, y_pred, direction_baseline=direction_baseline)
    return model, metrics, y_pred

def evaluate(y_true, y_pred, direction_baseline=None) -> dict:
    from scipy.stats import spearmanr
    from sklearn.metrics import mean_absolute_error, mean_squared_error,
    ↪ r2_score
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    ic, _ = spearmanr(y_true, y_pred)
    if direction_baseline is None:
        direction_acc = float(np.mean(np.sign(y_true) == np.sign(y_pred)))
    else:
        baseline = np.asarray(direction_baseline)
        direction_acc = float(np.mean(np.sign(y_true - baseline) ==
        ↪ np.sign(y_pred - baseline)))
    return {
        "mse": float(mean_squared_error(y_true, y_pred)),
        "mae": float(mean_absolute_error(y_true, y_pred)),
        "r2": float(r2_score(y_true, y_pred)),
        "ic": float(ic) if not np.isnan(ic) else 0.0,
        "direction_acc": direction_acc,
        "n_test": int(len(y_true)),
    }

def compute_shap(model, X_test) -> tuple[np.ndarray, pd.DataFrame]:
    """Compute SHAP values and feature importance."""
    explainer = shap.TreeExplainer(model)
    shap_values = explainer.shap_values(X_test)
    importance = pd.DataFrame({
        "feature": X_test.columns,
        "importance": np.abs(shap_values).mean(axis=0),
    }).sort_values("importance", ascending=False).reset_index(drop=True)
    return shap_values, importance

def compute_lgbm_gain_importance(model: lgb.LGBMRegressor, feature_names:
    ↪ list[str]) -> pd.DataFrame:
    """Return LightGBM feature importance sorted by split gain."""
    gain = model.booster_.feature_importance(importance_type="gain")
    return (
        pd.DataFrame({"feature": feature_names, "importance": gain})
        .sort_values("importance", ascending=False)
        .reset_index(drop=True)
    )

def bidirectional_modeling(df: pd.DataFrame) -> tuple[dict, dict, pd.DataFrame]:
    """Forward: H3→return. Backward: return→H3. Return metrics and
    ↪ comparison."""
    df = build_features(df)
    feat_cols_h3 = [c for c in FEATURE_COLS if c in df.columns]
    feat_cols_ret = []
    for lag in range(1, config.MAX_LAG + 1):
        col = f"ret_lag{lag}"
        df[col] = df["return"].shift(lag)
        feat_cols_ret.append(col)
    ret_past = df["return"].shift(1)
    for w in [3, 5]:
        df[f"ret_roll_mean_{w}"] = ret_past.rolling(w, min_periods=w).mean()
        df[f"ret_roll_std_{w}"] = ret_past.rolling(w, min_periods=w).std()
        feat_cols_ret.extend([f"ret_roll_mean_{w}", f"ret_roll_std_{w}"])
    feat_cols_ret += ["month", "quarter"]

    # Each direction gets its own dropna to maximize usable samples

```

```

fwd_valid = df.dropna(subset=feat_cols_h3 + ["return"])
bwd_valid = df.dropna(subset=feat_cols_ret + ["H3t"]).copy()
bwd_valid["H3_prev"] = bwd_valid["H3t"].shift(1)
bwd_valid = bwd_valid.dropna(subset=["H3_prev"])

if len(fwd_valid) < 10 or len(bwd_valid) < 10:
    return {}, {}, pd.DataFrame()

# Forward: predict return from H3 features
Xf = fwd_valid[feat_cols_h3].values
yf = fwd_valid["return"].values
split_f = int(len(fwd_valid) * config.TRAIN_RATIO)
model_f, metrics_f, pred_f = train_lgbm(
    pd.DataFrame(Xf[:split_f], columns=feat_cols_h3),
    yf[:split_f],
    pd.DataFrame(Xf[split_f:], columns=feat_cols_h3),
    yf[split_f:],
)

# Backward: predict H3 from return features
Xb = bwd_valid[feat_cols_ret].values
yb = bwd_valid["H3t"].values
split_b = int(len(bwd_valid) * config.TRAIN_RATIO)
model_b, metrics_b, pred_b = train_lgbm(
    pd.DataFrame(Xb[:split_b], columns=feat_cols_ret),
    yb[:split_b],
    pd.DataFrame(Xb[split_b:], columns=feat_cols_ret),
    yb[split_b:],
    direction_baseline=bwd_valid["H3_prev"].values[split_b:],
)

# Feature importance for both directions
fi_forward = compute_lgbm_gain_importance(model_f, feat_cols_h3)
fi_backward = compute_lgbm_gain_importance(model_b, feat_cols_ret)

comparison = pd.DataFrame([
    {"direction": "H3→return (forward)", **metrics_f, "best_lag":
     ↳ _best_lag_from_importance(fi_forward, "H3_lag")},
    {"direction": "return→H3 (backward)", **metrics_b, "best_lag":
     ↳ _best_lag_from_importance(fi_backward, "ret_lag")},
])
return metrics_f, metrics_b, comparison

def _best_lag_from_importance(fi: pd.DataFrame, prefix: str) -> int:
    """Extract best lag number from feature importance DataFrame."""
    lag_feats = fi[fi["feature"].str.startswith(prefix)]
    if lag_feats.empty:
        return 0
    best = lag_feats.iloc[0]["feature"]
    return int(best.replace(prefix, "").replace("_lag", "").replace("lag", ""))

# -----
# Save outputs
# -----

def save_analysis_outputs(
    feature_data: pd.DataFrame,
    forward_pred: np.ndarray,
    backward_comparison: pd.DataFrame,
    shap_values: np.ndarray,
    shap_importance: pd.DataFrame,
    lgbm_importance: pd.DataFrame,
    forward_metrics: dict,
    backward_metrics: dict,
    test_data: pd.DataFrame,
    modeling: pd.DataFrame,
) -> None:
    config.OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    # Feature engineering dataset
    feature_data.to_csv(config.OUTPUT_DIR / "feature_engineering.csv",
     ↳ index=False, encoding="utf-8-sig")
    # Forward prediction results
    pred_df = test_data[["week", "H3t", "return"]].copy()
    pred_df["predicted_return"] = forward_pred
    pred_df["residual"] = pred_df["return"] - pred_df["predicted_return"]
    pred_df.to_csv(config.OUTPUT_DIR / "lgbm_forward_results.csv", index=False,
     ↳ encoding="utf-8-sig")

```

```

# Backward comparison
backward_comparison.to_csv(config.OUTPUT_DIR /
    ↳ "bidirectional_comparison.csv", index=False, encoding="utf-8-sig")
# LightGBM split-gain importance and SHAP importance are separate
↳ diagnostics.
lgbm_importance.to_csv(config.OUTPUT_DIR / "lgbm_feature_importance.csv",
    ↳ index=False, encoding="utf-8-sig")
# SHAP importance
shap_importance.to_csv(config.OUTPUT_DIR / "shap_importance.csv",
    ↳ index=False, encoding="utf-8-sig")
# Summary
summary = {"forward": forward_metrics, "backward": backward_metrics}
pd.DataFrame(summary).to_csv(config.OUTPUT_DIR / "lgbm_metrics.csv",
    ↳ encoding="utf-8-sig")

```

A.5 plots.py

```

"""Plot generation for experiment 1 outputs."""

from __future__ import annotations

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from . import config

def _setup_font() -> None:
    plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial",
    ↳ Unicode MS", "DejaVu Sans"]
    plt.rcParams["axes.unicode_minus"] = False

def plot_weekly_sentiment(weekly: pd.DataFrame) -> Path:
    _setup_font()
    path = config.OUTPUT_DIR / "weekly_sentiment_counts.png"
    fig, ax = plt.subplots(figsize=(12, 5))
    ax.bar(weekly["week"], weekly["WeekPositive"], label="正面", color="#2f9e44")
    ax.bar(weekly["week"], weekly["WeekNeutral"],
    ↳ bottom=weekly["WeekPositive"], label="中性", color="#868e96")
    ax.bar(weekly["week"], weekly["WeekNegative"],
    ↳ bottom=weekly["WeekPositive"] + weekly["WeekNeutral"], label="负面",
    ↳ color="#d9480f")
    ax.set_title(" 周度新闻情绪数量")
    ax.set_xlabel(" 周末交易日")
    ax.set_ylabel(" 新闻数量")
    ax.legend(ncol=3)
    fig.autofmt_xdate()
    fig.tight_layout()
    fig.savefig(path, dpi=180)
    plt.close(fig)
    return path

def plot_herd_index(herd: pd.DataFrame) -> Path:
    _setup_font()
    path = config.OUTPUT_DIR / "herd_index_timeseries.png"
    fig, ax = plt.subplots(figsize=(12, 5))
    ax.plot(herd["week"], herd["H1t"], label="H1 情绪偏离", color="#1971c2")
    ax.plot(herd["week"], herd["H2t"], label="H2 分歧度", color="#f08c00")
    ax.plot(herd["week"], herd["H3t"], label="H3 羊群强度", color="#c92a2a",
    ↳ linewidth=2)
    ax.set_title(" 周度羊群效应指标")
    ax.set_xlabel(" 周末交易日")
    ax.legend(ncol=3)
    fig.autofmt_xdate()
    fig.tight_layout()
    fig.savefig(path, dpi=180)
    plt.close(fig)
    return path

def plot_market_relation(modeling: pd.DataFrame) -> Path:

```

```

_setup_font()
path = config.OUTPUT_DIR / "herd_vs_hs300_return.png"
fig, ax1 = plt.subplots(figsize=(12, 5))
ax2 = ax1.twinx()
ax1.plot(modeling["week"], modeling["H3t"], color="#c92a2a", label="H3
↳ 羊群强度", linewidth=2)
ax2.plot(modeling["week"], modeling["return"], color="#1864ab",
↳ label="沪深300周收益率", linewidth=1.8)
ax1.set_title("羊群效应指标与沪深 300 周收益率")
ax1.set_xlabel("周末交易日")
ax1.set_ylabel("H3")
ax2.set_ylabel("周收益率")
lines = ax1.get_lines() + ax2.get_lines()
ax1.legend(lines, [line.get_label() for line in lines], loc="upper left")
fig.autofmt_xdate()
fig.tight_layout()
fig.savefig(path, dpi=180)
plt.close(fig)
return path

def plot_feature_importance(importance_df: pd.DataFrame) -> Path:
_setup_font()
path = config.OUTPUT_DIR / "feature_importance.png"
top = importance_df.head(10).iloc[::-1]
fig, ax = plt.subplots(figsize=(8, 5))
ax.barh(top["feature"], top["importance"], color="#1971c2")
ax.set_title("LightGBM Gain 特征重要性 (Top 10)")
ax.set_xlabel("Gain")
fig.tight_layout()
fig.savefig(path, dpi=180)
plt.close(fig)
return path

def plot_shap_dependence(shap_values: np.ndarray, X_test: pd.DataFrame,
↳ best_feature: str) -> Path:
_setup_font()
path = config.OUTPUT_DIR / "shap_dependence.png"
if best_feature not in X_test.columns:
    best_feature = X_test.columns[0]
feat_idx = list(X_test.columns).index(best_feature)
fig, ax = plt.subplots(figsize=(7, 5))
ax.scatter(X_test[best_feature].values, shap_values[:, feat_idx],
↳ alpha=0.7, color="#c92a2a", edgecolors="k", linewidths=0.3)
ax.set_xlabel(best_feature)
ax.set_ylabel(f"SHAP value ({best_feature})")
ax.set_title(f"SHAP 依赖图: {best_feature}")
z = np.polyfit(X_test[best_feature].values, shap_values[:, feat_idx], 1)
x_line = np.linspace(X_test[best_feature].min(),
↳ X_test[best_feature].max(), 100)
ax.plot(x_line, np.polyval(z, x_line), "--", color="#495057", alpha=0.7,
↳ label="趋势线")
ax.legend()
fig.tight_layout()
fig.savefig(path, dpi=180)
plt.close(fig)
return path

def plot_residuals(test_data: pd.DataFrame) -> Path:
_setup_font()
path = config.OUTPUT_DIR / "residual_timeseries.png"
fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(test_data["week"], test_data["residual"], color="#495057",
↳ linewidth=1.2)
ax.axhline(0, color="#c92a2a", linestyle="--", alpha=0.7)
ax.set_title("测试集残差时序图")
ax.set_xlabel("周末交易日")
ax.set_ylabel("残差 (实际 - 预测)")
fig.autofmt_xdate()
fig.tight_layout()
fig.savefig(path, dpi=180)
plt.close(fig)
return path

def plot_bidirectional_comparison(comparison: pd.DataFrame) -> Path:

```

```

_setup_font()
path = config.OUTPUT_DIR / "bidirectional_comparison.png"
metrics = ["mse", "mae", "r2"]
labels = ["MSE", "MAE", "R2"]
if comparison.empty or "direction" not in comparison.columns:
    fig, ax = plt.subplots(figsize=(7, 5))
    ax.text(0.5, 0.5, "数据不足, 无法绘制双向对比", ha="center", va="center",
           ↪ fontsize=14)
    ax.set_axis_off()
    fig.savefig(path, dpi=180)
    plt.close(fig)
    return path
fwd_row = comparison[comparison["direction"].str.contains("forward")]
bwd_row = comparison[comparison["direction"].str.contains("backward")]
if fwd_row.empty or bwd_row.empty:
    fig, ax = plt.subplots(figsize=(7, 5))
    ax.text(0.5, 0.5, "数据不足, 无法绘制双向对比", ha="center", va="center",
           ↪ fontsize=14)
    ax.set_axis_off()
    fig.savefig(path, dpi=180)
    plt.close(fig)
    return path
fwd = fwd_row[metrics].iloc[0].values
bwd = bwd_row[metrics].iloc[0].values
x = np.arange(len(metrics))
w = 0.35
fig, ax = plt.subplots(figsize=(7, 5))
ax.bar(x - w / 2, fwd, w, label="H3→ 收益 (正向)", color="#1971c2")
ax.bar(x + w / 2, bwd, w, label="收益 →H3 (反向)", color="#c92a2a")
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.set_title("双向建模效果对比")
ax.legend()
for i, (f, b) in enumerate(zip(fwd, bwd)):
    ax.text(i - w / 2, f + 0.001, f"{f:.4f}", ha="center", va="bottom",
           ↪ fontsize=9)
    ax.text(i + w / 2, b + 0.001, f"{b:.4f}", ha="center", va="bottom",
           ↪ fontsize=9)
fig.tight_layout()
fig.savefig(path, dpi=180)
plt.close(fig)
return path

def generate_all_plots(
    weekly: pd.DataFrame,
    herd: pd.DataFrame,
    modeling: pd.DataFrame,
    shap_values: np.ndarray,
    X_test: pd.DataFrame,
    shap_importance: pd.DataFrame,
    lgbm_importance: pd.DataFrame,
    test_pred_df: pd.DataFrame,
    comparison: pd.DataFrame,
) -> list[str]:
    best_feature = shap_importance.iloc[0]["feature"] if len(shap_importance) >
    ↪ 0 else "H3_lag1"
    paths = [
        plot_weekly_sentiment(weekly),
        plot_herd_index(herd),
        plot_market_relation(modeling),
        plot_feature_importance(lgbm_importance),
        plot_shap_dependence(shap_values, X_test, best_feature),
        plot_residuals(test_pred_df),
        plot_bidirectional_comparison(comparison),
    ]
    return [str(p.relative_to(config.ROOT)) for p in paths]

```

A.6 main.py

```

"""Executable pipeline for Experiment 1.

```

```

Run from repository root:

```

```

    python -m experiment1.main
"""

```

```

from __future__ import annotations

import argparse
import io
import json
import sys

import pandas as pd

if sys.stdout.encoding and sys.stdout.encoding.lower().replace("-", "") !=
↳ "utf8":
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding="utf-8",
↳ errors="replace")
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding="utf-8",
↳ errors="replace")

from . import config
from .analysis import (
    build_features,
    build_herd_index,
    build_modeling_dataset,
    bidirectional_modeling,
    compute_lgbm_gain_importance,
    compute_shap,
    save_analysis_outputs,
    train_lgbm,
    temporal_train_test_split,
)
from .analysis import FEATURE_COLS
from .data import (
    build_hs300_weekly,
    build_weekly_sentiment,
    clean_and_label_news,
    load_hs300_daily,
    load_news,
    make_trading_calendar,
    save_outputs,
    write_data_description,
)
from .plots import generate_all_plots

def _json_safe(obj):
    if hasattr(obj, "item"):
        return obj.item()
    if isinstance(obj, (pd.Timestamp,)):
        return str(obj)
    return str(obj)

def write_report(summary: dict) -> None:
    config.REPORT_DIR.mkdir(parents=True, exist_ok=True)
    a = summary["audit"]
    fwd = summary["forward_metrics"]
    bwd = summary["backward_metrics"]
    bcomp = pd.DataFrame(summary["bidirectional_comparison"]) if
↳ summary["bidirectional_comparison"] else pd.DataFrame()
    best_feat = summary.get("best_shap_feature", "H3_lag1")

    # Forward best lag
    fwd_best = bcomp[bcomp["direction"].str.contains("forward")].iloc[0] if
↳ len(bcomp) > 0 else None
    bwd_best = bcomp[bcomp["direction"].str.contains("backward")].iloc[0] if
↳ len(bcomp) > 0 else None

    report = f"""# 实验一：金融非结构化数据预处理与羊群效应预测分析报告

学生: {config.STUDENT_NAME}
学号: {config.STUDENT_ID}

## 1. 作业要求理解

老师提供的《金融数据挖掘实验指导书 20260617》以"实验一"为文件夹名发布，
↳ 但指导书正文实际包含三个连续环节：实验一生成周度情绪指标，实验二基于情绪指标构建羊群效应指数，
↳ 实验三将羊群效应指数与沪深300周收益对齐并完成 LightGBM 非线性预测建模。三部分前后依赖，
↳ 因此本次按完整链路完成。

关键口径如下：

```

```

- 新闻时间范围: {config.START_DATE} 至 {config.END_DATE}。
- 周度单位: 按沪深300交易日历每 {config.TRADING_DAYS_PER_WEEK} 个交易日作为一周,
  ↳ 先剔除非交易日新闻。
- 情绪标注: 使用 `yiyanghust/finbert-tone-chinese` BERT 模型本地 GPU 推理三分类。
- 情绪输出: `week`、`WeekPositive`、`WeekNeutral`、`WeekNegative`、`P_t`。
- 羊群指标: `H1t = P_t - E(P_t)`、`H2t = 1 -`
  ↳ `|Positive-Negative|/(Positive+Negative)`、`H3t = Norm(|H1t|) * (1 -`
  ↳ `Norm(H2t))`。
- 市场预测: 采用 LightGBM 非线性时序建模, 滞后 1~5 期特征 + 滚动统计 + 时间特征, 前 80% 训练、
  ↳ 后 20% 测试, 辅以 SHAP 可解释性分析和双向传导验证。

```

2. 数据清洗与情绪标注

原始新闻共 {a['raw_news_rows']} 行。经过日期范围、正文非空、正文去重、交易日过滤后,
 ↳ 进入周度统计的新闻为 {a['labeled_trading_day_rows']} 行, 样本交易日范围为
 ↳ {a['date_start']} 至 {a['date_end']}。

情绪标注采用 HuggingFace 开源中文金融情感模型 `yiyanghust/finbert-tone-chinese`
 ↳ (BERT/Transformer 架构), 本地 GPU 批量推理实现三分类 (正面/中性/负面)。标注方法为:
 ↳ {a.get('labeling_method', 'bert')}。

周度情绪表已写入 `outputs/experiment1/weekly\_sentiment.csv`, SQLite 表
 ↳ `experiment1.db::weekly\_sentiment`。

3. 羊群效应指标

`P\_t` 表示正面新闻占正负情绪新闻的比例。`E(P\_t)` 使用过去 4 个周度样本的滚动均值作为正常情绪基准。
 ↳ `H1t` 衡量本期情绪相对历史基准的偏离, `H2t` 衡量正负观点分歧程度, `H3t`
 ↳ 综合情绪异常程度与单边一致性。

羊群指标结果已写入 `outputs/experiment1/weekly\_herd\_index.csv`, 图表
 ↳ `outputs/experiment1/herd\_index\_timeseries.png`。

4. LightGBM 非线性预测建模

4.1 特征工程

对 H3t 构造滞后 1~5 期特征 (`H3\_lag1` ~ `H3\_lag5`)、3 周和 5 周滚动均值/标准差、
 ↳ 月份和季度时间特征。

4.2 时序划分

严格按时间顺序: 前 80% 训练, 后 20% 测试, 杜绝数据泄露。

4.3 正向建模: H3 → 收益率

项目	数值
MSE	{fwd['mse']:.6f}
MAE	{fwd['mae']:.6f}
R ²	{fwd['r2']:.4f}
IC (Spearman 秩相关)	{fwd.get('ic', 0):.4f}
方向准确率	{fwd.get('direction_acc', 0):.2%}
测试集样本数	{fwd['n_test']}

****关于 R² 为负的说明**:** R² < 0 表示模型预测精度不及简单均值预测, 这在小样本 (测试集仅
 ↳ {fwd['n_test']} 条) 时序预测中属正常现象。IC 和方向准确率更能反映模型的实用预测价值。

正向最优预测滞后 (SHAP 特征重要性): {fwd_best['best_lag']} if fwd_best is not None else
 ↳ 'N/A'}

4.4 SHAP 可解释性分析

SHAP 最重要特征: {best_feat}。图表见 `outputs/experiment1/shap\_dependence.png`。

4.5 双向传导验证

方向	MSE	MAE	R ²	IC	方向准确率	最优滞后
H3→收益率	{fwd['mse']:.6f}	{fwd['mae']:.6f}	{fwd['r2']:.4f}	{fwd.get('ic', 0):.4f}	{fwd.get('direction_acc', 0):.2%}	{fwd_best['best_lag']} if fwd_best is not None else 'N/A'}
收益率→H3	{bwd['mse']:.6f}	{bwd['mae']:.6f}	{bwd['r2']:.4f}	{bwd.get('ic', 0):.4f}	{bwd.get('direction_acc', 0):.2%}	{bwd_best['best_lag']} if bwd_best is not None else 'N/A'}

4.6 金融反身性分析

```
{summary.get('reflexivity_notes', '正向模型 (H3→收益率) 和反向模型 (收益率→H3)
↪ 的对比揭示了舆情与市场之间的双向传导关系。')}
```

5. 输出文件索引

```
- `experiment1/assignment.md`: 指导书正文按原内容抽取的 Markdown。
- `data/experiment1/original/`: 老师发布的 Word 与 zip 原件。
- `data/experiment1/raw/`: 解压后的新闻数据与沪深 300 价格数据。
- `data/experiment1/数据说明.md`: 数据来源、格式和处理口径。
- `outputs/experiment1/weekly_sentiment.csv`: 实验一周度情绪指标。
- `outputs/experiment1/weekly_herd_index.csv`: 实验二羊群效应指标。
- `outputs/experiment1/modeling_dataset.csv`: 建模对齐数据。
- `outputs/experiment1/feature_engineering.csv`: 特征工程数据集。
- `outputs/experiment1/lgbm_forward_results.csv`: LightGBM 正向预测结果。
- `outputs/experiment1/bidirectional_comparison.csv`: 双向建模对比。
- `outputs/experiment1/shap_importance.csv`: SHAP 特征重要性。
- `outputs/experiment1/lgbm_metrics.csv`: 模型评估指标。
- `outputs/experiment1/experiment1.db`: SQLite 数据库存储结果。
- `outputs/experiment1/*.png`: 周度情绪、羊群指标、收益对比、特征重要性、SHAP 依赖、残差、
↪ 双向对比图。
```

6. 运行方式

在仓库根目录执行:

```
```powershell
python -m experiment1.main
```
"""
    path = config.REPORT_DIR / "experiment1_report.md"
    path.write_text(report, encoding="utf-8")
    (config.OUTPUT_DIR / "experiment1_report.md").write_text(report,
    ↪ encoding="utf-8")

def run() -> dict:
    config.OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    print(" 读取老师配套数据...")
    hs300 = load_hs300_daily()
    calendar = make_trading_calendar(hs300)
    news = load_news()

    print(" 清洗新闻并标注三分类情绪...")
    labeled, audit = clean_and_label_news(news, calendar)
    weekly_sentiment = build_weekly_sentiment(labeled)

    print(" 构建羊群效应指标...")
    weekly_herd = build_herd_index(weekly_sentiment)
    hs300_weekly = build_hs300_weekly(hs300, calendar)
    modeling = build_modeling_dataset(weekly_herd, hs300_weekly)

    print(" 特征工程...")
    featured = build_features(modeling)
    feature_cols = [c for c in FEATURE_COLS if c in featured.columns]
    featured_valid = featured.dropna(subset=feature_cols +
    ↪ ["return"]).reset_index(drop=True)

    print(" 时序划分 + LightGBM 训练...")
    train_df, test_df = temporal_train_test_split(featured_valid)
    X_train = train_df[feature_cols]
    y_train = train_df["return"]
    X_test = test_df[feature_cols]
    y_test = test_df["return"]

    model, forward_metrics, y_pred = train_lgbm(X_train, y_train, X_test,
    ↪ y_test)
    print(f" 正向 R²={forward_metrics['r2']:.4f},
    ↪ MSE={forward_metrics['mse']:.6f}")

    print("SHAP 可解释性分析...")
    lgbm_importance = compute_lgbm_gain_importance(model, feature_cols)
    shap_values, shap_importance = compute_shap(model, X_test)
    best_shap_feature = shap_importance.iloc[0]["feature"] if
    ↪ len(shap_importance) > 0 else "H3_lag1"
```

```

print(" 双向传导验证...")
_, _, bidir_comparison = bidirectional_modeling(modeling)
backward_metrics = {}
if len(bidir_comparison) > 0:
    bwd = bidir_comparison[bidir_comparison["direction"].str.contains("backward")
    if len(bwd) > 0:
        backward_metrics = {
            "mse": float(bwd.iloc[0]["mse"]),
            "mae": float(bwd.iloc[0]["mae"]),
            "r2": float(bwd.iloc[0]["r2"]),
            "ic": float(bwd.iloc[0].get("ic", 0)),
            "direction_acc": float(bwd.iloc[0].get("direction_acc", 0)),
            "n_test": int(bwd.iloc[0]["n_test"]),
        }

print(" 保存 CSV、SQLite 数据库和图表...")
save_outputs(labeled, weekly_sentiment, weekly_herd, hs300_weekly, modeling)
# Build test prediction df for plots and saving
test_pred_df = test_df[["week", "H3t", "return"]].copy()
test_pred_df["predicted_return"] = y_pred
test_pred_df["residual"] = test_pred_df["return"] -
    test_pred_df["predicted_return"]

save_analysis_outputs(
    featured, y_pred, bidir_comparison, shap_values, shap_importance,
    lgbm_importance,
    forward_metrics, backward_metrics, test_pred_df, modeling,
)

plots = generate_all_plots(
    weekly_sentiment, weekly_herd, modeling,
    shap_values, X_test, shap_importance, lgbm_importance,
    test_pred_df, bidir_comparison,
)
write_data_description(audit)

summary = {
    "audit": audit,
    "weekly_rows": len(weekly_sentiment),
    "modeling_rows": len(modeling),
    "feature_rows": len(featured_valid),
    "forward_metrics": forward_metrics,
    "backward_metrics": backward_metrics,
    "bidirectional_comparison": bidir_comparison.to_dict(orient="records"),
    "best_shap_feature": best_shap_feature,
    "lgbm_gain_top5": lgbm_importance.head(5).to_dict(orient="records"),
    "shap_top5": shap_importance.head(5).to_dict(orient="records"),
    "plots": plots,
    "reflexivity_notes": _build_reflexivity_notes(forward_metrics,
    backward_metrics, bidir_comparison),
}
(config.OUTPUT_DIR / "summary.json").write_text(
    json.dumps(summary, ensure_ascii=False, indent=2, default=json_safe),
    encoding="utf-8",
)
write_report(summary)
print(f" 完成。输出目录: {config.OUTPUT_DIR}")
return summary

def _build_reflexivity_notes(fwd, bwd, comp) -> str:
    if not fwd or not bwd:
        return ""
    fwd_r2 = fwd.get("r2", 0)
    bwd_r2 = bwd.get("r2", 0)
    fwd_ic = fwd.get("ic", 0)
    bwd_ic = bwd.get("ic", 0)
    fwd_da = fwd.get("direction_acc", 0)
    bwd_da = bwd.get("direction_acc", 0)

    # Both R2 negative — model worse than mean prediction
    if fwd_r2 < 0 and bwd_r2 < 0:
        notes = (
            f" 双向模型 R2 均为负 (正向 {fwd_r2:.4f}, 反向 {bwd_r2:.4f}), "
            " 说明在当前小样本条件下, 两个方向的预测精度均不及简单均值预测。\\n\\n"
        )
    # Still compare IC for directional signal
    if abs(fwd_ic) > abs(bwd_ic):

```

```

        notes += (
            f" 但从 IC (秩相关) 来看, 正向 IC={fwd_ic:.4f}, 反向 IC={bwd_ic:.4f}, "
            " 情绪对收益的方向性预测信号略强于反向传导。"
        )
    elif abs(bwd_ic) > abs(fwd_ic):
        notes += (
            f" 但从 IC (秩相关) 来看, 反向 IC={bwd_ic:.4f}, 正向 IC={fwd_ic:.4f}, "
            " 收益对情绪的方向性引导信号略强。"
        )
    else:
        notes += " 双向 IC 相近, 情绪与收益之间的方向性传导信号对等。"
    return notes

if fwd_r2 > bwd_r2:
    return (
        f" 正向模型  $R^2$ ={fwd_r2:.4f} 高于反向  $R^2$ ={bwd_r2:.4f}, "
        " 说明羊群情绪对收益率的预测能力强于收益率对情绪的引导能力, "
        " 舆情具有一定的先行指标价值。"
    )
elif bwd_r2 > fwd_r2:
    return (
        f" 反向模型  $R^2$ ={bwd_r2:.4f} 高于正向  $R^2$ ={fwd_r2:.4f}, "
        " 说明市场收益率对羊群情绪的引导能力强于情绪对收益的预测, "
        " 反映'收益驱动舆情'的反身性特征。"
    )
else:
    return (
        f" 双向模型  $R^2$  相近 (正向 {fwd_r2:.4f}, 反向 {bwd_r2:.4f}), "
        " 情绪与收益之间存在对等的双向传导关系。"
    )

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="实验一: 金融新闻情绪、
    ↪ 羊群效应和LightGBM预测检验")
    return parser.parse_args()

def main() -> None:
    parse_args()
    run()

if __name__ == "__main__":
    main()

```